



青岛科技大学

Qingdao University of Science & Technology

数据结构与算法分析2

梁宏涛

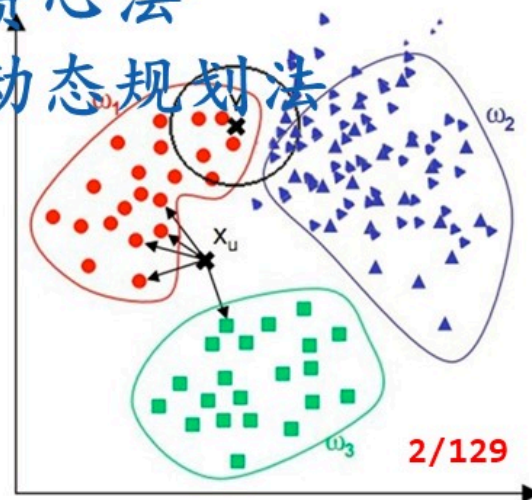
lht@qust.edu.cn

周二&周四上午

1/129

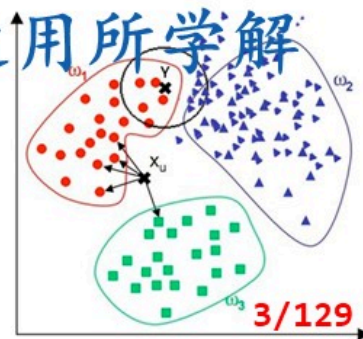
— 教学提纲

- 必修，3.5学分
- 理论教学48学时
 - 绪论4
 - 递归算法设计技术4
 - 穷举法4
 - 分治法4
 - 回溯法6
 - 分支限界法6
 - 贪心法6
 - 动态规划法8
 - NP完全问题2
- 实践教学16学时
 - 实验1常见数据结构及应用
 - 实验2基本算法设计方法
 - 实验3分治法
 - 实验4回溯法
 - 实验5分支限界法
 - 实验6贪心法
 - 实验7动态规划法



— 课程简介

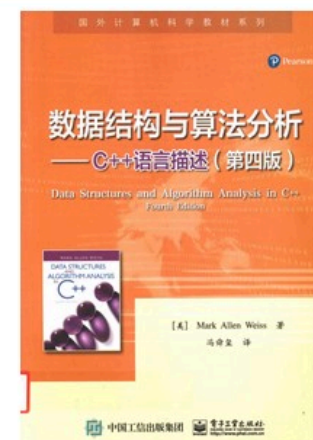
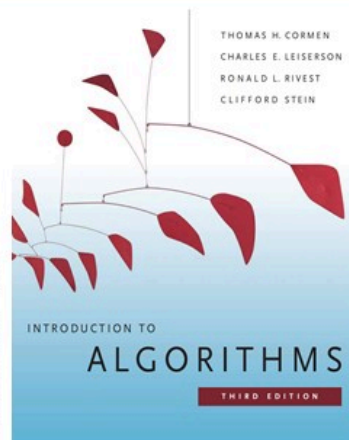
- 本课程是计算机科学与技术专业的专业课。通过课堂讲授、课堂练习和讨论互动、课后作业和上机实验等教学手段，系统掌握算法的有关概念和算法设计的基本策略。
- 使学生掌握计算机算法的基本概念和特性，了解计算机相关学科中算法设计技术的重要性，掌握算法时间复杂性的分析方法。
- 结合典型示例和实战问题求解过程，使学生重点掌握分治法、回溯法、分支限界法、贪心法和动态规划法等算法设计策略，具备灵活运用所学解决复杂工程应用问题的能力





参考教材:

- 《算法设计与分析》（第3版），李春葆等 清华大学出版社 2024
- 《算法设计与分析（第3版）学习指导》，李春葆等 清华大学出版社 2024
- 《算法设计与分析（第3版）在线编程实验指导》，
- 《图解算法》，Anditya著，袁国忠译，人民邮电出版社，2017
- 《Introduction Algorithms》（Third Edition）
- 《数据结构与算法分析》C++语言描述（第四版）



第1章—绪论

提纲 CONTENTS

1.1 算法概述

1.2 算法分析



1.3 算法设计工具—STL

5/129

以下对算法概念の説明中_____是错误的。

- ☐ A 算法是程序的灵魂
- ☐ B 程序=算法+数据结构
- ☐ C 算法是求解一个问题方法的描述
- ☒ D 算法是将输入加工成输出的过程

算法的特性包含：

- ☒ A 有限性
- ☒ B 正确性
- ☒ C 可行性
- ☐ D 输入性
- ☐ E 输出性

以下___可以作为描述算法的方法之一。

- ☒ A 自然语言
- ☒ B 流程图
- ☒ C 伪代码
- ☒ D 程序

单选题 1分

算法设计的基本步骤有：①分析求解问题，②描述算法，③选择数据结构和算法设计策略，④证明算法正确性，⑤算法分析。以下正确的顺序是：

A ①, ②, ③, ④, ⑤

B ①, ③, ②, ④, ⑤

C ①, ②, ③, ⑤, ④

D ①, ③, ②, ⑤, ④

9/129

单选题 1分

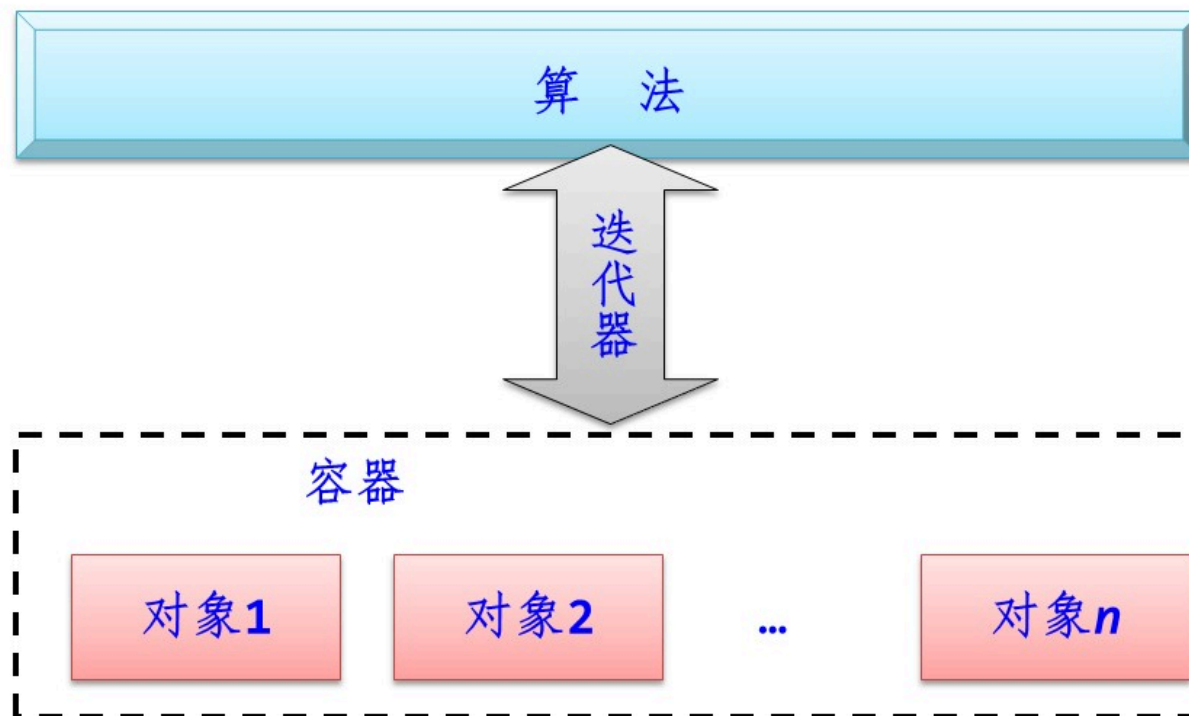
算法设计时是从零开始——“万丈高楼平地起”，还是“工欲善其事必先利其器”？

- ☐ A 一步一步写，从零开始
- ☒ B 工欲善其事必先利其器，借助工具

10/129

1.3 算法设计工具-STL

1.3.1 STL概述



11/129

1.什么是STL容器

数据结构容器	说 明	头文件
vector	向量	<vector>
string	字符串	<string>
deque	双端队列	<deque>
list	链表	<list>
stack	栈	<stack>
queue	队列	<queue>
priority_queue	优先队列	<queue>
set/multiset	集合/多重集合	<set>
map/multimap	映射/多重映射	<map>
unordered_set	哈希集合	<unordered_set>
unordered_map	哈希映射	unordered_map>

12/129

2.什么是STL算法

- STL提供了大约**100**个通用算法，例如，**sort()**用于序列数据的排序，**find()**用于序列数据的查找等。
- 这些算法采用模板函数设计（即泛型设计），具有很好的通用性。
- 在复杂算法设计中灵活使用它们可以**简化实现细节，提高算法设计效率**。

通用算法	说 明	头文件
<code>accumulate(beg,end,initv)</code>	返回 <code>[beg,end)</code> 范围内的元素和再加上 <code>initv</code>	<code><numeric></code>
<code>binary_search(beg,end,val)</code>	在 <code>[beg,end)</code> 指定的有序序列中二分查找 <code>val</code>	<code><algorithm></code>
<code>lower_bound(beg,end,val)</code>	在 <code>[beg,end)</code> 指定的有序序列中二分查找第一个大于等于 <code>val</code> 的位置	<code><algorithm></code>
<code>upper_bound(beg,end,val)</code>	在 <code>[beg,end)</code> 指定的有序序列中二分查找第一个大于 <code>val</code> 的位置	<code><algorithm></code>
<code>max(a,b)</code>	返回 <code>a</code> 和 <code>b</code> 中最大值	<code><algorithm></code>
<code>min(a,b)</code>	返回 <code>a</code> 和 <code>b</code> 中最小值	<code><algorithm></code>
<code>swap(a,b)</code>	交换 <code>a</code> 和 <code>b</code>	<code><algorithm></code>
<code>reverse(beg,end)</code>	逆置 <code>[beg,end)</code> 范围内的元素	<code><algorithm></code>

14/129

通用算法	说 明	头文件
<code>prev_permutation(beg, end)</code>	重置 <code>[beg, end)</code> 标记的排列为上一个排列	<code><algorithm></code>
<code>next_permutation(beg, end)</code>	重置 <code>[beg, end)</code> 标记的排列为下一个排列	<code><algorithm></code>
<code>sort(beg, end, [comp])</code>	根据比较关系 <code>comp</code> 对 <code>[beg, end)</code> 范围内的元素排序	<code><algorithm></code>
<code>sort(beg, end, [comp])</code>	根据比较关系 <code>comp</code> 对 <code>[beg, end)</code> 范围内的元素采用稳定方法排序	<code><algorithm></code>
<code>find(beg, end, val)</code>	在 <code>[beg, end)</code> 范围内查找 <code>val</code> 元素	<code><algorithm></code>
<code>find(beg, end, val, pred)</code>	在 <code>[beg, end)</code> 范围内查找满足 <code>pred</code> 谓词的元素	<code><algorithm></code>

15/129

3.什么是STL迭代器

- STL迭代器用于访问容器中的数据对象。
- 迭代器像C/C++中的指针，算法通过迭代器来定位和操作容器中的元素。
- 使用*操作符来获取数据。

迭代器的常用运算如下：

- **++**：正向移动迭代器。
- **--**：反向移动迭代器。
- *****：返回迭代器所指的元素值。

例如，以下语句定义一个含3个整数的vector容器myv:

```
vector<int> myv={1,2,3};
```

如果要正向输出所有元素，可以使用正向迭代器:

```
vector<int>::iterator it;           //定义正向迭代器it
for (it=myv.begin();it!=myv.end();++it) //从头到尾遍历元素
    printf("%d ",*it);             //输出: 1 2 3
printf("\n");
```

或者

```
for(int x:myv)
    printf("%d ",x);               //输出: 1 2 3
printf("\n");
```

18/129

例如，以下语句定义一个含**3**个整数的**vector**容器**myv**:

```
vector<int> myv={1,2,3};
```

如果要反向输出所有元素，可以使用反向迭代器:

```
vector<int>::reverse_iterator rit;           //定义反向迭代器rit
for (rit=myv.rbegin();rit!=myv.rend();++rit) //从尾到头遍历
    printf("%d ",*rit);                      //输出: 3 2 1
printf("\n");
```

1.3.2 vector (向量容器)

- 向量容器相当于动态数组（当超过空间大小时通常按两倍大小扩展），用于存储具有相同数据类型的序列元素。
- 可以从末尾快速的插入与删除元素。
- 快速地随机访问元素。
- 在序列中间插入、删除元素较慢，因为需要移动插入或删除处后面的所有元素。

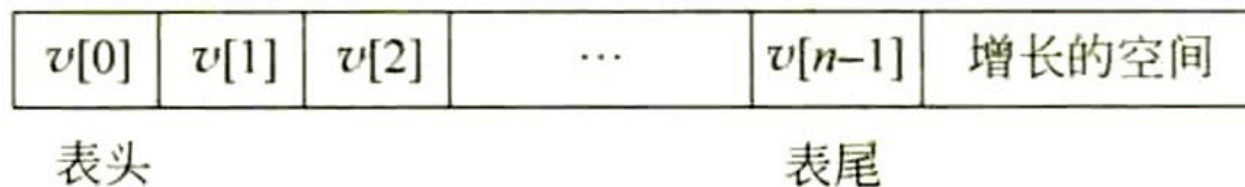


图 1.11 vector 容器 v 的存储方式

_3/129

定义vector容器的几种方式

<code>vector<int> v1;</code>	<code>//定义元素为int的向量v1</code>
<code>vector<int> v2(10);</code>	<code>//指定v2的初始大小为10个int元素</code>
<code>vector<int> v3(10, -1);</code>	<code>//指定v3的初始大小为10,</code> <code>且10元素值均为-1</code>
<code>vector<int> v4(a, a+5);</code>	<code>//用数组a[0..4]共5个元素</code> <code>初始化v4</code>
<code>vector<int> v5(v1);</code>	<code>//由v1复制产生v5</code>

21/129



【例1.6】数组中的第 k 个最大元素（LeetCode215★★）。设计一个算法求无序数组`nums`中第 k （ $1 \leq k \leq$ 数组的长度）大的整数，注意不是第 k 个不同的元素，而是数组排序后的第 k 个最大的元素。

例如，`nums={3, 2, 1, 5, 6, 4}`， $k=2$ ，`nums`递减排序后是`{6, 5, 4, 3, 2, 1}`，答案为5，若 $k=5$ ，答案为2。

要求设计如下成员函数：

```
int findKthLargest(vector<int>& nums, int k) { }
```

主观题 10分

数组中的第 k 个最大元素（**LeetCode215★★**）。设计一个算法求无序数组`nums`中第 k （ $1 \leq k \leq$ 数组的长度）大的整数，注意不是第 k 个不同的元素，而是数组排序后的第 k 个最大的元素。

例如，`nums={3, 2, 1, 5, 6, 4}`， $k=2$ ，`nums`递减排序后是`{6, 5, 4, 3, 2, 1}`，答案为5，若 $k=5$ ，答案为2。

要求设计如下成员函数：

```
int findKthLargest(vector<int>& nums, int k) { }
```

23/129

解法1

- 将nums中全部元素递增排序。
- 则nums[n-k]就是第k个最大元素。

```
class Solution {  
public:  
    int findKthLargest(vector<int>& nums,int k) {  
        int n=nums.size();  
        sort(nums.begin(),nums.end());  
        return nums[n-k];  
    }  
};
```

EXE 执行用时为**112ms**，内存消耗为**44.4MB**。

24/129

解法2

- 将nums中全部元素递减排序。
- 则nums[k-1]就是第k个最大元素。

```
class Solution {  
public:  
    int findKthLargest(vector<int>& nums,int k) {  
        int n=nums.size();  
        sort(nums.begin(),nums.end(),greater<int>());  
        return nums[k-1];  
    }  
};
```

EXE 执行用时为**92ms**，内存消耗为**44.5MB**。

25/129

vector主要的成员函数

- **empty()**: 判断容器是否为空。
- **size()**: 返回容器的中的实际元素个数。
- **reserve(n)**: 为容器预分配 n 个元素的存储空间。
- **capacity()**: 返回容器在重新进行内存分配以前所能容纳的元素个数。
- **resize(n)** : 调整容器的大小, 使其能容纳 n 个元素。
- **[k]**: 返回容器中下标为 k 的元素。
- **at(k)**: 与 $[k]$ 相同。
- **front()**: 获取容器中的首元素。
- **back()**: 获取容器中的尾元素。
- **push_back()**: 在容器尾部添加了一个元素。

vector主要的成员函数(续)

- **pop_back()**: 删除容器尾部的元素。
- **insert(pos, e)**: 在**pos**位置插入元素**e**。
- **erase()**: 删除容器中某个迭代器或者迭代器区间指定的元素。
- **clear()**: 删除容器中所有元素。
- **begin()**: 返回容器中首元素的迭代器。
- **end()**: 返回容器中尾元素后面一个位置的迭代器。
- **rbegin()**: 返回容器中尾元素的迭代器。
- **rend()**: 返回容器中首元素前面一个位置的迭代器。

sort的应用

- 在许多应用中需要对数据排序，排序有多种方式，如递增排序、递减排序以及按某个成员排序等。
- 可以利用**STL通用算法sort()**实现各种方式的排序，只是该算法针对的是采用数组（如数组、**vector**或者**deque**容器）存储的数据，不适合链表（如**list**容器）数据。
- **sort()**通用算法使用十分灵活，下面以**vector**为例按照其元素类型分别讨论。

1) 内置数据类型的排序

- 对于内置数据类型的数据，**sort()**默认是以**less<T>**（小于关系函数）作为关系比较函数实现递增排序，为了实现递减排序，需要调用大于关系函数**greater<T>**。

```
vector<int> myv={2,1,5,4,3};           //myv={2,1,5,4,3}
sort(myv.begin(),myv.end(),less<int>()); //指定递增排序
sort(myv.begin(),myv.end(),greater<int>()); //指定递减排序
sort(myv.begin(),myv.end());           //不指定less<int>,
                                         默认为递增排序
```

Lambda表达式形式:

```
sort(myv.begin(),myv.end(),[](int a,int b)->bool { return  
a<b; });          //递增排序（默认）
```

```
sort(myv.begin(),myv.end(),[](int a,int b)->bool { return  
a>b; });          //递减排序
```


2) 自定义置数据类型的排序

方式1: 在声明结构体类型中重载<运算符, 以实现按指定成员的递增或者递减排序。如`sort(myv.begin(), myv.end())`调用`Stud`结构体中默认<运算符对`myv`容器的所有元素实现排序。

方式2: 自己在结构体或类中重载函数调用运算符()以实现按指定成员的递增或者递减排序。如

`sort(myv.begin(), myv.end(), Cmp())`调用结构体`Cmp`的()运算符对`myv`容器的所有元素实现排序。


```

struct Stud {                                //学生结构体类型
    int no;                                  //学号
    string name;                             //姓名
    Stud(int no1,string name1) {             //构造函数
        no=no1;
        name=name1;
    }
    bool operator<(const Stud &s) const {    //方式1: 重载<运算符
        return no>s.no;                     //用于按no递减排序
    }
};

struct Cmp {                                //方式2: 定义关系函数()
    bool operator()(const Stud &s,const Stud &t) const {
        return s.name<t.name;              //用于按name递增排序
    }
};

```

32/129

```

int main() {
    Stud a[]={Stud(2,"Mary"),Stud(1,"John"),Stud(5,"Smith")};
    int n=sizeof(a)/sizeof(a[0]);
    vector<Stud> myv(a,a+n);
                                //myv: 2,Mary 1,John 5,Smith
    sort(myv.begin(),myv.end());
                                //默认使用<运算符, 按no递减排序
    sort(myv.begin(),myv.end(),Cmp());
                                //使用Cmp中的()运算符, 按name递增排序

    return 0;
}

```

Lambda表达式形式:

```
sort(myv.begin(),myv.end(),[](Stud& a,Stud& b)->bool { return  
a.no<b.no; });           //按no递增排序
```

```
sort(myv.begin(),myv.end(),[](Stud& a,Stud& b)->bool { return  
a.name>b.name; });       //按name递减排序
```

1.3.3 string (字符串容器)

- **string**是一个保存字符序列的容器，它的所有元素为字符类型，类似**vector<char>**。
- 除了字符串的一些常用操作以外，还包含了所有的序列容器的操作。字符串的常用操作包括增加、删除、修改、查找比较、连接、输入、输出等。
- **string** 重载了许多运算符，包括 **+**, **+=**, **<**, **=**, **[]**, **<<**和**>>**等。



【例1.7】 字符串中的单词数（**LintCode1243★**）。给定一个字符串**s**，其中一个单词定义为不含空格的连续字符串，单词用空格分隔，设计一个算法求**s**中的单词数。

例如**s="Hello, my name is John"**，答案为**5**，对应的**5**个单词是**"Hello"**、**"my"**、**"name"**、**"is"**和**"John"**。

要求设计的成员函数如下：

```
int countSegments(string &s) { }
```

解

- 用 **ans** 存放单词数（初始时 **ans** 置为 0）。
- 用 **i** 遍历 **s**，先跳过开头的空格，向后查找第一个空格位置 **j**，如果 **j** $\neq$ -1 说明找到了空格，则该空格之前有一个单词，**ans** 增 1，再跳过连续的空格继续上述过程。
- 循环结束后若 **i** 遍历完时返回 **ans**，否则说明最后还有一个单词，返回 **ans+1**。

任务2

【例1.7】字符串中的单词数（**LintCode1243★**）。给定一个字符串**s**，其中一个单词定义为不含空格的连续字符串，单词用空格分隔，设计一个算法求**s**中的单词数。

例如**s="Hello, my name is John"**，答案为5，对应的5个单词是**"Hello"**、**"my"**、**"name"**、**"is"**和**"John"**。

要求设计的成员函数如下：

```
int countSegments(string &s) { }
```

```

class Solution {
public:
    int countSegments(string &s) {
        int n=s.length();
        if(n==0) return 0;
        int ans=0; //存放答案
        int i=0;
        while(i<n && s[i]==' ') i++; //跳过开头的空格
        int j=s.find(" ",i); //从i查找后面的空格
        while (j!=-1) {
            ans++; //找到一个单词后ans增1
            i=j+1;
            while(i<n && s[i]==' ') i++; //跳过中间的空格
            j=s.find(" ",i); //从i查找下一个空格
        }
        if(i==n) return ans; //i遍历完时返回ans
        else return ans+1; //i未遍历完时返回ans+1
    }
};

```

EXE

执行用时为**41ms**，内存消耗为**4.13MB**。

39/129

定义string容器的几种方式

string(): 建立一个空的字符串。

string(const string& str): 用字符串**str**建立当前字符串。

string(const string& str, size_type i): 用字符串**str**起始于**i**的字符建立当前字符串。

string(const string& str, size_type i, size_type len): 用字符串**str**起始于**i**的**len**个字符建立当前字符串。

string(const char* cstr): 用C-字符串**cstr**建立当前字符串。

string(const char* chars, size_type len): 用C-字符串**cstr**的开头**len**个字符建立当前字符串。

string(size_type len, char c): 用**len**个字符**c**建立当前字符串。

string主要的成员函数

- **empty()**: 判断当前字符串是否为空串。
- **size()**: 返回当前字符串的实际字符个数（返回结果为**size_type**类型）。
- **length()**: 返回当前字符串的实际字符个数。
- **compare(const string& str)**: 返回当前字符串与字符串**str**的比较结果。在比较时，若两者相等，返回**0**；前者小于后者，返回**-1**；否则返回**1**。
- **[k]**: 返回当前字符串中下标为**k**的字符。
- **at(k)**: 与**[k]**相同。
- **front()**: 获取当前字符串的首字符。
- **back()**: 获取当前字符串中的尾字符。
- **push_back()**: 在当前字符串尾部添加了一个字符。
- **pop_back()**: 删除当前字符串尾部的字符。
- **append(str)**: 在当前字符串的末尾添加一个字符串**str**。

41/129

string主要的成员函数（续）

- **insert(size_type i, const string& str):** 在当前字符串的*i*处插入一个字符串*str*。
- **find(string& s, size_type pos=0):** 在当前字符串中*pos*位置开始查找字符串*s*的第一个位置，找到后返回其位置，没有找到返回-1。
- **replace(size_type i, size_type len, const string& str):** 将当前字符串中起始于*i*的*len*个字符用一个字符串*str*替换。
- **replace(iterator beg, iterator end, const string& str):** 将当前字符串中[beg, end)区间的所有字符用字符串*str*替换。
- **substr(size_type i):** 返回当前字符串起始于*i*的子串。
- **substr(size_type i, size_type len):** 返回当前字符串起始于*i*的长度为*len*的子串。
- **clear():** 删除当前字符串中的所有的字符。
- **erase(size_type i):** 删除当前字符串的从*i*开始的所有字符。
- **erase(size_type i, size_type len):** 删除当前字符串的从*i*开始的*len*个字符。

42/129

1.3.4 deque (双端队列容器)

- **deque**是一个双端队列类模板。
- 所谓双端队列就是将其中的所有元素看成是一个序列，这样有两个端点（分别称为队头端和队尾端），支持在两端插入和删除元素操作。
- 双端队列容器不像**vector**把所有的元素保存在一块连续的内存块，而是采用多个连续的存储块存放数据元素。所以扩容时性能较好。

定义deque容器的几种方式

```
deque<int> dq1;           //定义元素为int的双端队列dq1
deque<int> dq2(10);        //指定dq2的初始大小为10个int元素
deque<int> dq3(10, -1);    //指定dq3的10个元素的初值均为-1
deque<int> dq4(dq2.begin(), dq2.end());
                          //用dq2的所有元素初始化dq4
```

deque主要的成员函数

- **empty()**: 判断双端队列容器是否为空队。
- **size()**: 返回双端队列容器中元素个数。
- **front()**: 取队头元素。
- **back()**: 取队尾元素。
- **push_front(e)**: 在队头插入元素 e 。
- **push_back(e)**: 在队尾插入元素 e 。
- **pop_front()**: 删除队头一个元素。
- **pop_back()**: 删除队尾一个元素。
- **erase()**: 从双端队列容器中删除一个或几个元素。
- **clear()**: 删除双端队列容器中所有元素。
- **begin()**: 返回容器中首元素的迭代器。
- **end()**: 返回容器中尾元素后面一个位置的迭代器。
- **rbegin()**: 返回容器中尾元素的迭代器。
- **rend()**: 返回容器中首元素前面一个位置的迭代器。

45/129



【例1.8】滑动窗口（POJ2823）。时间限制：12000ms，空间限制：65536KB。

给定一个大小为 n （ $n \leq 10^6$ ）的整数数组 a ，有一个长度为 k 的滑动窗口从左向右滑过数组 a ，每次移动一个元素，求该滑动窗口中最小元素和最大元素的序列。

例如， $a=\{1, 3, -1, -3, 5, 3, 6, 7\}$ ， $k=3$ ，得到最小元素序列是 $\{-1, -3, -3, -3, 3, 3\}$ ，最大元素的序列是 $\{3, 3, 5, 5, 6, 7\}$ 。

输入格式：输入由两行组成，第一行是两个整数 n 和 k ，第二行包含 n 个整数，整数之间用空格分隔。

输出格式：输出两行，第一行是滑动窗口最小元素序列，第二行是滑动窗口最大元素序列。

输入样例：

8 3

1 3 -1 -3 5 3 6 7

输出样例：

-1 -3 -3 -3 3 3

3 3 5 5 6 7

解

1	3	-1	-3	5	3	6	7	最小元素：-1，最大元素：3
1	3	-1	-3	5	3	6	7	最小元素：-3，最大元素：3
1	3	-1	-3	5	3	6	7	最小元素：-3，最大元素：5
1	3	-1	-3	5	3	6	7	最小元素：-3，最大元素：5
1	3	-1	-3	5	3	6	7	最小元素：3，最大元素：6
1	3	-1	-3	5	3	6	7	最小元素：3，最大元素：7

48/129

任务3

【例1.8】滑动窗口（POJ2823）。时间限制：12000ms，空间限制：65536KB。

给定一个大小为 n （ $n \leq 10^6$ ）的整数数组 a ，有一个长度为 k 的滑动窗口从左向右滑过数组 a ，每次移动一个元素，求该滑动窗口中最小元素和最大元素的序列。

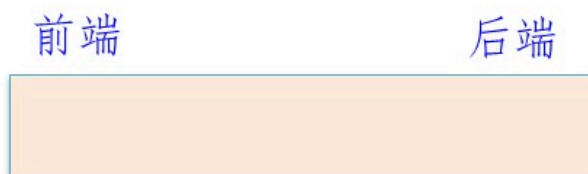
例如， $a=\{1, 3, -1, -3, 5, 3, 6, 7\}$ ， $k=3$ ，得到最小元素序列是 $\{-1, -3, -3, -3, 3, 3\}$ ，最大元素的序列是 $\{3, 3, 5, 5, 6, 7\}$ 。

- 考虑求滑动窗口最大元素序列，定义一个双端队列`maxdq`，用队头元素表示当前滑动窗口内最大元素的位置（下标），
- 用数组`maxans`存放所有滑动窗口中的最大元素（长度为 n 的数组中长度为 k 的滑动窗口共 $n-k+1$ 个）。
- 用 i 遍历 a ，对于数组元素 $a[i]$ ：
 - ① 若队列`maxdq`为空，将 i 从队尾进队。
 - ② 若队列`maxdq`不空，将 $a[i]$ 和队尾元素 x （ $x = \text{nums}[\text{maxdq.back}()]$ ）进行比较，若 $a[i]$ 大于 x ，将 x 从队尾出队，直到 $a[i]$ 小于等于队尾元素或队列为空，再将 i 从队尾进队。

- 对于当前滑动窗口，若队头元素位置`maxdq.front()`“过期”，也就是满足 $i - \text{maxdp.front()} \geq k$ ，则将队头元素从队头出队。
- 当 $i \geq k - 1$ 时将新的队头元素作为滑动窗口的最大值添加的`maxans`中。
- 求滑动窗口最小元素序列`minans`时定义一个双端队列`mindq`，操作过程与上述过程类似。

以 (4,3,5,4,3,3,6,7) 序列求最大序列为例 ($k=3$)

nums: 0[4] 1[3] 2[5] 3[4] 4[3] 5[3] 6[6] 7[7] $k=3$



ans: 5 5 5 4 6 7



结果是

(5, 5, 5, 4, 6, 7)

```
#include<iostream>
#include<vector>
#include<deque>
using namespace std;
#define MAXN 1000005
int maxans[MAXN],minans[MAXN]; //最大序列和最小序列

int main() {
    int n,k;
    scanf("%d%d",&n,&k);
    vector<int> a;    //向量a, 存放n个数据
    int x;
    for(int i=0;i<n;i++) {
        scanf("%d",&x);
        a.push_back(x);
    }
```

```

deque<int> maxdq;
deque<int> mindq;
int m=0;
for(int i=0;i<n;i++) {    //遍历a中元素
    while(!maxdq.empty() && a[i]>a[maxdq.back()])
        maxdq.pop_back();    //将队尾小于a[i]的元素从队尾出队
    maxdq.push_back(i);    //将元素下标i进队尾
    if(i-maxdq.front()>=k) //将队头过期的元素从队头出队
        maxdq.pop_front();
    while(!mindq.empty() && a[i]<a[mindq.back()])
        mindq.pop_back();    //将队尾大于a[i]的元素从队尾出队
    mindq.push_back(i);    //将元素下标i进队尾
    if(i-mindq.front()>=k) //将队头过期的元素从队头出队
        mindq.pop_front();
    if(i>=k-1) {    //i>=k-1时对应一个窗口
        maxans[m]=a[maxdq.front()];    //队头添加到maxans
        minans[m++]=a[mindq.front()]; //队头添加到minans
    }
}

```

54/129

```
for(int i=0;i<m-1;i++)
    printf("%d ",minans[i]);
printf("%d\n",minans[m-1]);
for(int i=0;i<m-1;i++)
    printf("%d ",maxans[i]);
printf("%d\n",maxans[m-1]);
return 0;
}
```

EXE 执行用时为**9516ms**，内存消耗为**10944KB**。

55/129

1.3.5 list (链表容器)

- **list**是一个双链表类模板，可以从任何地方快速插入与删除。采用循环双链表实现。
- 不能随机访问元素，为了访问表容器中特定的元素，必须从表头开始，沿着指针从一个元素到下一个元素，直到找到要找的元素。
- **list**容器插入元素比**vector**快，另外对每个元素单独分配空间，所以不存在空间不够而需要重新分配的情况。

56/129

定义list容器的几种方式

<code>list<int> l1;</code>	//定义元素为int的链表l1
<code>list<int> l2 (10);</code>	//指定链表l2的初始大小为10个int元素
<code>list<int> l3 (10, -1);</code>	//指定l3的10个初始元素的初值均为-1
<code>list<int> l4(a, a+5);</code>	//用数组a[0..4]共5个元素初始化l4

list主要的成员函数

- **empty()**: 判断链表容器是否为空。
- **size()**: 返回链表容器中实际元素个数。
- **push_back()**: 在链表尾部插入元素。
- **pop_back()**: 删除链表容器的最后一个元素。
- **remove ()**: 删除链表容器中所有指定值的元素。
- **remove_if(cmp)**: 删除链表容器中满足条件的元素。
- **erase()**: 从链表容器中删除一个或几个元素。
- **unique()**: 删除链表容器中相邻的重复元素。
- **clear()**: 删除链表容器中所有的元素。

list主要的成员函数(续)

- **insert(pos, e):** 在pos位置插入元素e, 即将元素e插入到迭代器pos指定元素之前。
- **insert(pos, n, e):** 在pos位置插入n个元素e。
- **insert(pos, pos1, pos2):** 在迭代器pos处插入[pos1, pos2)的元素。
- **reverse():** 反转链表。
- **sort():** 对链表容器中的元素排序。
- **begin():** 返回容器中首元素的迭代器。
- **end():** 返回容器中尾元素后面一个位置的迭代器。
- **rbegin():** 返回容器中尾元素的迭代器。
- **rend():** 返回容器中首元素前面一个位置的迭代器。



```
list<int> lst; //定义list容器lst
list<int>::iterator it,it1,it2;
lst.push_back(5); //添加5个整数5,2,4,1,3
lst.push_back(2);
lst.push_back(4);
lst.push_back(1);
lst.push_back(3);
it=lst.begin(); //it指向首元素5
it1=++lst.begin(); //it1指向第2个元素2
it2=--lst.end(); //it2指向尾元素3
```

60/129

1.3.6 stack (栈容器)

- **stack**是一个栈类模板，和数据结构中的栈一样，具有后进先出的特点。
- **stack**属于适配器容器，所谓适配器容器是指基于其他容器而实现的容器，也就是说适配器容器包含另一个容器作为其底层容器，在底层容器基础上实现适配器容器的功能。
- **stack**默认的底层容器是**deque**，也可以底层容器为**vector**或者**list**。

61/129

定义stack容器的几种方式

<code>stack<int> st1;</code>	<code>//默认底层容器为deque</code>
<code>stack<int,vector<int>> st2;</code>	<code>//指定底层容器为vector</code>
<code>stack<int,list<int>> st3;</code>	<code>//指定底层容器为list</code>

stack主要的成员函数

- **empty()**: 判断栈容器是否为空。
- **size()**: 返回栈容器中实际元素个数。
- **push(e)**: 元素 e 进栈。
- **top()**: 返回栈顶元素。
- **pop()**: 元素出栈。



在调用**top()**和**pop()**之前务必通过**empty()**检测保证栈是非空的，否则出现不可预测的错误。



【例1.9】字符串解码（LeetCode394★★）。给定一个经过编码的有效字符串 s ，设计一个算法返回 s 解码后的字符串。编码规则是用“ $k[e]$ ”表示方括号内的字符串 e （仅包含小写字母）正好重复 k （ k 保证为正整数） 次。

例如， $s="3[a]2[bc]"$ ，答案为 $"aaabcbcb"$ ，若 $s="abc3[cd]xyz"$ ，答案为 $"abccdcddcdxyz"$ 。

要求设计如下成员函数：

```
string decodeString(string s) { }
```

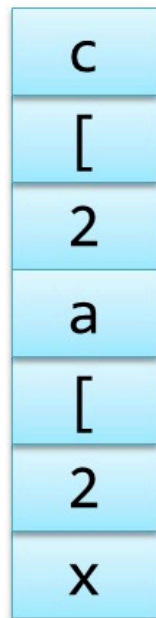
*/129

解

- 对于有效字符串 s ，将其看成 $s = s_1 + \cdots + s_{i-1} + s_i$ ，其中每个 s_i 是连续字母串或者形如 " $k[e]$ " 的有效字符串（称为有效子串）。
- 例如， $s = \text{"ab2[5[c]]3[de]"}$ ，可以将 s 看成由 3 个有效子串构成即 $s = \text{"ab"} + \text{"2[5[c]]"} + \text{"3[de]"}$ ，将每个 s_i 解码为 d_i ，那么 s 的解码字符串 $\text{ans} = d_1 + \cdots + d_{i-1} + d_i$ 。

- 那么如何解码"**k[n[e]]**"呢？
- 关键是需要考虑方括号的匹配，其中第一个']'与第二个 '[' 匹配，其中第二个']'与第一个 '[' 匹配，也就是说这里的方括号采用遵循**最近匹配**原则，所以采用一个字符串栈**st**。
- 每次遇到']'就解码一次。
- 依次将解码串 d_1 到 d_i 进栈，最后依次出栈并逆序连接构成 $ans=d_1+\cdots+d_i$ ，返回**ans**即可。

例如, $s = "x2[a2[c]]"$



栈st



栈st



栈st

xaccacc


```

class Solution {
public:
    string decodeString(string s) {
        stack<string> st;                //定义一个栈st
        int i=0,n=s.size();
        while (i<n) {                    //遍历s
            if (isdigit(s[i])) {        //遇到数字
                string tmp="";
                while(i<n && isdigit(s[i])) { //提取整数串tmp
                    tmp+=s[i++];
                }
                st.push(tmp);            //将整数串tmp进栈
            }
        }
    }
}

```

68/129

```
else if(isalpha(s[i])) {           //遇到字母
    string tmp="";
    while(i<n && isalpha(s[i])) {   //提取字母串tmp
        tmp+=s[i++];
    }
    st.push(tmp);                  //将字母串tmp进栈
}
else if(s[i]=='[')                 //遇到 '[' 时
    st.push(string(1,s[i++]));     //将 '[' 转换为 "[" 后进栈
```

69/129

else {	//遇到']'时
string e="";	
while (st.top()!="[") {	//取[e]中的e
e=st.top()+e;	//同级字符串连接（逆向）
st.pop();	
}	
st.pop();	//出栈"["
string d="";	
int k=stoi(st.top()); st.pop();	//提取数字k
for (int i=0;i<k;i++)	//将"k[e]"解码为d
d+=e;	
st.push(d);	//将d进栈
i++;	//跳过']'
}	
}	

70/129

```
string ans="";  
while(!st.empty()) { //由栈st中所有串连接的逆序构成ans  
    ans=st.top()+ans;  
    st.pop();  
}  
return ans;  
}  
};
```

EXE 执行用时为**0ms**，内存消耗为**6.4MB**。

71/129

1.3.7 queue (队列容器)

- **queue**是一个队列类模板，和数据结构中的队列一样，具有先进先出的特点。
- **queue**也是适配器容器，其默认的底层容器是**deque**，也可以指定底层容器为**list**。
- **queue**不允许顺序遍历，没有提供**begin()/end()**和**rbegin()/rend()**这样的用于迭代器的成员函数。

72/129

定义queue容器的几种方式

```
queue<int> qu1;
```

//默认底层容器为**deque**

```
queue<int,list<int>> qu2;
```

//指定底层容器为**list**

queue主要的成员函数

- **empty()**: 判断队列容器是否为空。
- **size()**: 返回队列容器中实际元素个数。
- **front()**: 返回队头元素。
- **back()**: 返回队尾元素。
- **push(e)**: 元素e进队。
- **pop()**: 元素出队。



在调用**front()**、**back()**和**pop()**之前务必通过**empty()**检测保证队列是非空的，否则出现不可预测的错误。

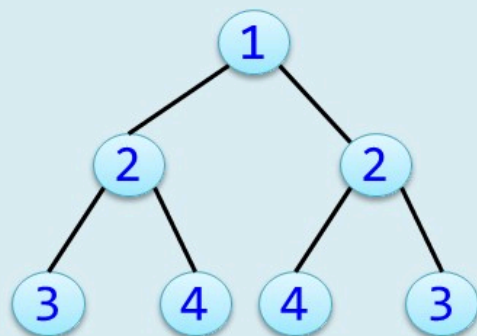


【例1.10】 对称二叉树（LeetCode101★）。给定一棵二叉树，设计一个算法检查它是否是镜像对称的，若是镜像对称的返回 **true**，否则返回 **false**。

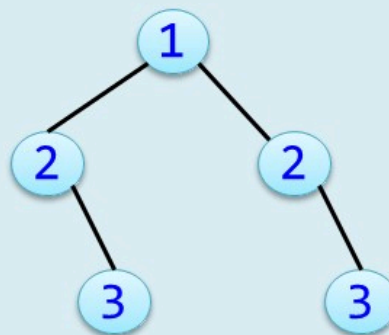
例如，如图(a)所示的二叉树A是镜像对称的，返回 **true**。而如图(b)所示的二叉树B不是镜像对称的，返回 **false**。

要求设计如下成员函数：

```
bool isSymmetric(TreeNode* root) { }
```



(a) 二叉树A



(b) 二叉树B

75/129

解

- 采用二叉树层次遍历方法求解。
- 分别同步层次遍历左右子树，先将左右孩子分别进队`qu1`和`qu2`。
- 在两个队列均不空时循环：`qu1`出队结点`p1`，`qu2`出队结点`p2`，若`p1`和`p2`中一个空另外一个不空时返回假，否则若`p1`和`p2`结点值不同时返回假，否则将`p1`的左右孩子（含空结点）依次进队`qu1`，将`p2`的右左孩子（含空结点）依次进队`qu2`。
- 循环结束后返回`true`。

76/129

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if (root==NULL)
            return true;
        queue<TreeNode*> qu1,qu2;           //定义两个队列
        TreeNode* p1,*p2;
        qu1.push(root->left);              //左孩子进qu1
        qu2.push(root->right);              //右孩子进qu2
    }
};
```

77/129

```

while (!qu1.empty() && !qu2.empty()) {
    p1=qu1.front(); qu1.pop();           //从qu1出队结点p1
    p2=qu2.front(); qu2.pop();           //从qu2出队结点p2
    if ((p1 && !p2) || (!p1 && p2))       //一空一不空返回假
        return false;
    if (p1!=NULL && p2!=NULL) {
        if (p1->val != p2->val)           //结点值不同返回假
            return false;
        qu1.push(p1->left);               //p1的左孩子进队qu1
        qu1.push(p1->right);              //p1的右孩子进队qu1
        qu2.push(p2->right);              //p2的右孩子进队qu2
        qu2.push(p2->left);               //p2的左孩子进队qu2
    }
}
return true;                             //循环结束后返回true
};

```

EXE

执行用时为**4ms**，内存消耗为**16.3MB**。

78/129

1.3.8 priority_queue (优先队列容器)

- **priority_queue**是一个优先队列类模板。
priority_queue也是适配器容器，其默认的底层容器是**vector**，也可以指定为**deque**容器。
- 优先队列中元素可以以任意顺序进队，但出队运算时只能出队最高优先级的元素。

priority_queue主要的成员函数

- **empty()**: 判断优先队列容器是否为空。
- **size()**: 返回优先队列容器中实际元素个数。
- **push(e)**: 元素 e 进队。
- **top()**: 获取队头元素。
- **pop()**: 元素出队。

1) 内置数据类型的优先队列

- 对于C/C++内置数据类型，默认是`less<T>`（小于关系函数）作为关系函数，值越大优先级的越高（即默认为大根堆）。
- 可以改为以`greater<T>`作为关系函数，这样值越大优先级的越低（即小根堆）。

```
priority_queue<int> pq1;                //默认大根堆  
priority_queue<int,vector<int>,greater<int>> pq2; //小根堆
```

Lambda表达式形式:

```
auto comp1=[](int a,int b) { return a<b; };  
    //大根堆比较关系  
priority_queue<int,deque<int>,decltype(comp1)> pq3(comp1);  
  
auto comp2=[](int a,int b) { return a>b; };  
    //小根堆比较关系  
priority_queue<int,deque<int>,decltype(comp2)> pq4(comp2);
```

82/129

2) 自定义数据类型的优先队列

方式1: 在声明结构体类型中重载<运算符以指定优先级, 如 `priority_queue<Stud> pq1`调用默认的<运算符创建堆pq1 (是大根堆还是小根堆由<重载函数体确定)。

方式2: 在声明结构体类型中重载>运算符以指定优先级, 如 `priority_queue<Stud, vector<Stud>, greater<Stud>> pq2`调用重载>运算符创建堆pq2, 此时需要指定优先队列的低层容器 (这里为vector, 也可以是deque)。

方式3: 自己在结构体或类中重载函数调用运算符()以指定元素的优先级, 如 `priority_queue<Stud, vector<Stud>, StudCmp> pq3`调用StudCmp结构体的()运算符创建堆pq3, 此时需要指定优先队列的低层容器 (这里为vector, 也可以是deque, 但不能是list)。



【例1.11】 数组中的第 k 个最大元素。题目描述见例1.6。这里采用优先队列求解。



【例1.6】 数组中的第 k 个最大元素（LeetCode215★★）。设计一个算法求无序数组`nums`中第 k （ $1 \leq k \leq$ 数组的长度）大的整数，注意不是第 k 个不同的元素，而是数组排序后的第 k 个最大的元素。

例如，`nums={3, 2, 1, 5, 6, 4}`， $k=2$ ，`nums`递减排序后是`{6, 5, 4, 3, 2, 1}`，答案为5，若 $k=5$ ，答案为2。

要求设计如下成员函数：

```
int findKthLargest(vector<int>& nums, int k) { }
```

解法1

- 采用大根堆的优先队列`maxpq`求解。
- 先将`nums`全部元素进队，再出队 k 次，最后出队的那个元素就是第 k 个最大的元素。

```

class Solution {
public:
    int findKthLargest(vector<int>& nums,int k) {
        priority_queue<int> maxpq;           //定义大根堆maxpq
        for(int i=0;i<nums.size();i++)      //全部元素进队
            maxpq.push(nums[i]);
        int ans;
        for(int i=0;i<k;i++) {               //出队k次
            ans=maxpq.top();
            maxpq.pop();
        }
        return ans;
    }
};

```



执行用时为**8ms**，内存消耗为**10.1MB**。

86/129

解法2

- 采用小根堆的优先队列`minpq`。
- 先将`nums`中前 k 个元素进队，再用 i 遍历`nums`的剩余元素，将大于堆顶元素`nums[i]`替换该堆顶元素（即出队后将`nums[i]`进队），始终保持堆中恰好有 k 个元素。
- 最后返回堆顶元素就是第 k 个最大的元素。


```

class Solution {
public:
    int findKthLargest(vector<int>& nums,int k) {
        priority_queue<int,vector<int>,greater<int>> minpq;
                                //小根堆
        for(int i=0;i<k;i++)    //进队前k个整数
            minpq.push(nums[i]);
        for(int i=k;i<nums.size();i++) {    //处理其他的整数
            if(nums[i]>minpq.top()) {
                minpq.pop();
                minpq.push(nums[i]);
            }
        }
        return minpq.top();    //返回堆顶整数
    }
};

```

EXE

执行用时为**8ms**，内存消耗为**9.7MB**。

88/129

1.3.9 set (集合容器) / multiset (多重集容器)

- **set/multiset**是集合类模板，都属于关联容器，所谓关联容器就是每个元素有一个**key**（关键字），通过**key**进行元素操作，关键字与元素位置没有对应关系。
- **set**中元素的关键字是唯一的，**multiset**中元素的关键字可以不唯一。
- 它们内部采用红黑树组织，默认情况下按关键字自动进行升序排列，所以查找、插入和删除运算的速度比较快（时间复杂度均为 $O(\log_2 n)$ ）。

定义set容器的几种方式

```
set<int> s1;  
int a[] = {10,20,30,40,50};  
set<int> s2(a,a+5);  
set<int> s3(s2);  
set<int> s4(s2.begin(), s2.end());  
struct Cmp {  
    bool operator() (const int&a, const int& b) const {  
        return a<b;  
    }  
};  
set<int,Cmp>s5;
```

90/129

set主要的成员函数

- **empty()**: 判断容器是否为空。
- **size()**: 返回容器中实际元素个数。
- **find(k)**: 若容器中存在关键字为 k 的元素, 返回该元素的迭代器, 否则返回**end()**。
- **count(k)**: 返回容器中关键字 k 出现的次数。
- **upper_bound()**: 返回一个迭代器, 指向第一个关键字大于 k 的元素。
- **lower_bound()**: 返回一个迭代器, 指向第一个关键字大于或者等于 k 的元素。
- **insert()**: 插入元素。
- **erase()**: 删除容器中的一个或几个元素。
- **clear()**: 删除容器中的所有元素。
- **begin()**: 用于正向迭代, 返回容器中首元素的迭代器。
- **end()**: 用于正向迭代, 返回容器中尾元素后面一个位置的迭代器。
- **rbegin()**: 用于反向迭代, 返回容器中尾元素的迭代器。
- **rend()**: 用于反向迭代, 返回容器中首元素前面一个位置的迭代器。

91/129



【例1.12】第三大的数（LeetCode414★）。给定一个非空数组 **nums**，设计一个算法求此数组中第三大的数（是指在所有不同整数中排第三大的数），如果不存在，则返回数组中最大的数。

例如，**nums**={3, 2, 1}，答案为1。

要求设计如下成员函数：

```
int thirdMax(vector<int>& nums) { }
```

解

- 建立一个`set<int>`类型的容器`s`，将`a`中全部元素插入到`s`中（自动除重）。
- 先让迭代器`it`指向最后元素（即最大元素）。如果`s`中元素个数少于3，返回`*it`，否则前移两次，再返回`*it`。

93/129

```

class Solution {
public:
    int thirdMax(vector<int>& nums) {
        set<int> s;
        for(int x:nums)           //将nums全部元素插入到s中
            s.insert(x);
        set<int>::iterator it;
        it=s.end();
        it--;                     //让it指向最大元素
        if(s.size()<3)
            return *it;
        it--; it--;              //前移2次
        return *it;              //返回第3大元素
    }
};

```



执行用时为**4ms**，内存消耗为**10.3MB**。

94/129

1.3.10 map (映射容器) / multimap (多重映射容器)

- **map/multimap**是映射类模板，同样属于关联容器即按关键字**key**进行元素操作。**map/multimap**中的元素属于**pair**结构体：

```
struct pair {  
    T first;  
    T second;  
}
```

- **pair**中有两个分量（二元组），**first**为第一个分量（在**map**中对应**key**），**second**为第二个分量（在**map**中对应**value**）。
- 同时**pair**对**==**、**!=**、**<**、**>**、**<=**、**>=**共**6**个运算符进行重载，提供了按照字典序对元素对进行大小比较的比较运算符模板函数。

- **map/multimap**内部也是用红黑树组织，默认情况下按关键字自动进行升序排列。
- 可以根据**key**快速找到与之对应的**value**（查找时间为 $O(\log_2 n)$ ）。
- **map**中不允许关键字重复出现，支持[]运算符，而**multimap**中允许关键字重复出现，但不支持[]运算符。

定义map容器的几种方式

```
map<char,int> mp1;  
mp1['a']=10;  
mp1['b']=30;  
mp1['c']=50;  
mp1['d']=70;  
map<char,int> mp2 (mp1.begin(),mp1.end());  
map<char,int> mp3 (mp2);  
struct Cmp {  
    bool operator() (const int&a, const int& b) const {  
        return a<b;  
    }  
};  
map<char,int,Cmp> mp4;
```

97/129

map主要的成员函数

- **empty()**: 判断容器是否为空。
- **size()**: 返回容器中实际元素个数。
- **map[k]**: 返回关键字为 k 的元素, 不存在时插入以 k 为关键字的元素。
- **find()**: 在容器中查找元素。
- **count()**: 容器中指定关键字的元素个数 (**map**中只有**1**或者**0**)。
- **insert(e)**: 插入一个元素 e 并返回该元素的位置。
- **erase()**: 删除容器中的一个或几个元素。
- **clear()**: 删除所有元素。
- **begin()**: 用于正向迭代, 返回容器中首元素的迭代器。
- **end()**: 用于正向迭代, 返回容器中尾元素后面一个位置的迭代器。
- **rbegin()**: 用于反向迭代, 返回容器中尾元素的迭代器。
- **rend()**: 用于反向迭代, 返回容器中首元素前面一个位置的迭代器。

98/129

【例1.13】有如下学生类型：

```
struct Stud {                                //Stud结构体
    int score;                               //分数
    string name;                             //姓名
    Stud(int s,string n) {                  //构造函数
        score=s;
        name=n;
    }
};
```

给定一个包含若干学生元素的数组**st**，设计一个算法求最高分数的学生姓名（可能有多个最高分数的学生）。

解

- 定义一个元素类型为`<int, vector<string>>`的map容器 `mymap`，其关键字为`int`类型的方式，值为具有相同方式的学生姓名即`vector<string>`类型。
- 这里求最高分的姓名，为此设计`Cmp`结构体指定`mymap`按分数递减排列。
- 遍历`st`创建`mymap`中的元素，最后返回首元素的值即可。

100/129

```

struct Cmp {                                //Cmp结构体
    bool operator()(const int& a,const int& b) const {
        return a>b;                        //指定mymap按关键字递减排列
    }
};

vector<string> Maxscore(vector<Stud>& st) {
    map<int,vector<string>,Cmp> mymap;
    for(Stud x:st) {                       //遍历st
        if(mymap.count(x.score)==0)        //没有找到
            mymap[x.score]={x.name};
        else {                             //找到了
            vector<string> tmp=mymap[x.score];
            tmp.push_back(x.name);
            mymap[x.score]=tmp;
        }
    }
    return mymap.begin()->second;
}

```

101/129

1.3.11 unordered_set (哈希集合容器)

- **unordered_set**与**set**类似，每个元素对应一个关键字，所有元素的关键字是唯一的。
- 只是**unordered_set**容器采用哈希表实现，利用拉链法解决冲突，所有元素是无序排列的，按关键字查找的时间复杂度接近 **$O(1)$** 。
- 定义**unordered_set**容器的方式与**set**类似，但不能指定关键字的排列方式。

102/129

unordered_set主要的成员函数

- **empty()**: 判断容器是否为空。
- **size()**: 返回容器的长度。
- **[k]**: 返回关键字 k 的元素的值。
- **at(k)**: 与**[k]**的功能相同。
- **count(k)**: 返回容器中关键字 k 出现的次数, 结果为**0**或者**1**。
- **find(k)**: 如果容器中存在关键字为 k 的元素, 返回其迭代器, 否则返回**end()**。
- **insert(e)**: 插入元素 e 。
- **erase()**: 从容器删除一个或几个元素。
- **clear()**: 删除所有元素。
- **begin()** 返回当前容器中首元素的迭代器。
- **end()** 返回当前容器中尾元素后一个位置的迭代器。

103/129

【例1.14】 给定一个无序的整数序列 v ，设计一个算法求其中不相同的元素个数。

解

- 定义一个 `unordered_set<int>` 容器 `s`，将 `v` 中所有元素插入到 `s` 中，返回 `s` 的长度即可。

```
int Count(vector<int>&v) {  
    unordered_set<int> s;  
    for(auto e:v)  
        s.insert(e);  
    return s.size();  
}
```

105/129

1.3.12 unordered_map (哈希映射容器)

- **unordered_map**与**map**类似，每个元素为**pair**类型，所有元素的关键字是唯一的。
- **unordered_map**容器采用哈希表实现，利用拉链法解决冲突，所有元素是无序排列的，按关键字查找的时间复杂度接近 **$O(1)$** 。
- 定义**unordered_map**容器的方式与**map**类似，但不能指定关键字的排列方式。

106/129

unordered_map主要的成员函数

- **empty()**: 判断容器是否为空。
- **size()**: 返回容器中实际元素个数。
- **map[k]**: 返回关键字为 k 的元素, 不存在时以 k 作为关键字插入一个元素。
- **at[k]**: 同 $ma[k]$ 。
- **find(k)**: 在容器中查找关键字为 k 的元素。
- **count(k)**: 返回容器中关键字为 k 的元素个数, 结果只有1或者0。
- **insert(e)**: 插入一个元素 e 并返回该元素的位置。
- **erase()**: 删除容器中的一个或几个元素。
- **clear()**: 删除容器中所有元素。
- **begin()**: 返回当前容器中首元素的迭代器。
- **end()**: 返回当前容器中尾元素后一个位置的迭代器。

107/129

【例1.15】 给定一个由小写字母组成的字符串 **s**，设计一个算法求其中所有唯一出现的字母，并且按照字母在 **s** 中出现的次序返回所有唯一出现字母构成的字符串。

解

- 设计一个 `unordered_map<char, int>` 类型的哈希映射 `cntmap` 用于字母计数，遍历 `s` 一次求出所有字母出现的次数。
- 用字符串 `ans` 存放答案（初始为空），再遍历 `s` 一次，将出现次数为 `1` 的字母添加到 `ans` 中。最后返回 `ans`。

109/129

```
string unique(string& s) {  
    unordered_map<char,int> cntmap;  
    for(char x:s) cntmap[x]++;  
    string ans;  
    for(char x:s) {  
        if(cntmap[x]==1) ans.push_back(x);  
    }  
    return ans;  
}
```

110/129



纸上得来终觉浅，
绝知此事要躬行。

2024年3月12日

111/14